

Code:

```
1  [org 0x0100]
2      jmp start
3  multiplicand: dd 1300
4  multiplier:   dw 500
5  result:       dd 0
6  start:        mov cl, 16
7                mov dx, [multiplier]
8  checkbit:     shr dx, 1
9                jnc skip
10               mov ax, [multiplicand]
11               add [result], ax
12               mov ax, [multiplicand+2]
13               adc [result+2], ax
14  skip:         shl word [multiplicand], 1
15               rcl word [multiplicand+2], 1
16               dec cl
17               jnz checkbit
18               mov ax, 0x4c00
19               int 0x21
```

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0113 Stack +0 0000 Flags 7200
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 0010 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 01F4 SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 0 0 0

CMD >
010D B110 MOV CL,10
010F BB160701 MOV DX,[0107]
0113 D1EA SHR DX,1
0115 730E JNC 0125
0117 A10301 MOV AX,[0103]
011A 01060901 ADD [0109],AX
011E A10501 MOV AX,[0105]
0121 11060801 ADC [0108],AX
0125 D1260301 SHL W[0103],1

```

First we moved the number of bits in CX

And the multiplier in DX.

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0115 Stack +0 0000 Flags 7214
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 0010 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 00FA SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 1 0

CMD >1
0113 D1EA SHR DX,1
0115 730E JNC 0125
0117 A10301 MOV AX,[0103]
011A 01060901 ADD [0109],AX
011E A10501 MOV AX,[0105]
0121 11060801 ADC [0108],AX
0125 D1260301 SHL W[0103],1
0129 D1160501 RCL W[0105],1
012D FEC9 DEC CL

```

Now we shift right the multiplier to know about the LSB from here we have two options:

- If cx stores 1 (lsb=1) add the multiplier into the result
- If stores 0 nothing has to be done simply shift left.

Above we can see carry flag has not been raised this shl and rcl operation will be performed.

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0125 Stack +0 0000 Flags 7214
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 0010 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 00FA SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 1 0

CMD >
0115 730E JNC 0125
0125 D1260301 SHL W[0103],1
0129 D1160501 RCL W[0105],1
012D FEC9 DEC CL
012F 75E2 JNZ 0113
0131 B8004C MOV AX,4C00
0134 CD21 INT 21
0136 7504 JNZ 013C
0138 B5C0 TEST AX,AX

```

Here the multiplicand is shifted to the left

And carry register is decremented.

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 012F Stack +0 0000 Flags 7214
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 000F BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 00FA SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 1 0

CMD >
012D FEC9 DEC CL
012F 75E2 JNZ 0113
0131 B804C MOV AX,4C00
0134 CD21 INT 21
0136 7504 JNZ 013C
0138 85C0 TEST AX,AX
013A 741C JZ 0158
013C C746DC000 MOV [BP-24],0000
0141 8E5EFC MOV DS,[BP-04]

2 0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.+.
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

Zero flag will be raised when CF=0 and
That would happen when all the bits would've
Been processed .

As only 1 bit has been processed this process is again repeated.

now if LSB is 1 the following process in followed:

```

DOSBox 0.74-3,...
AX 1450 SI 0000 CS 19F5 IP 011A Stack +0 0000 Flags 7211
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 000E BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 003E SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 0 1

CMD >
0117 A10301 MOV AX,[0103]
011A 01060901 ADD [0109],AX
011B A10501 MOV AX,[0105]
0121 11060B01 ADC [010B],AX
0125 D1260301 SHL W,[0103],1
0129 D1160501 RCL W,[0105],1
012D FEC9 DEC CL
012F 75E2 JNZ 0113
0131 B804C MOV AX,4C00

2 0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.+.
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

Multiplier is moved into AX which is
Then less significant word is added
into AX again

The multiplicand is added to the result using extended 32bit addition.

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0121 Stack +0 0000 Flags 7204
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 000E BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 003E SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 0 0

CMD >
011F A10501 MOV AX,[0105]
0121 11060B01 ADC [010B],AX
0125 D1260301 SHL W,[0103],1
0129 D1160501 RCL W,[0105],1
012D FEC9 DEC CL
012F 75E2 JNZ 0113
0131 B804C MOV AX,4C00
0134 CD21 INT 21
0136 7504 JNZ 013C

2 0 1 2 3 4 5 6 7 8 9 A B C D E F
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.+.
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 C0 11
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

Now the more significant word is added
Into ax.

After all the 16 bits are processed :



```
DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 012F Stack +0 0000 Flags 7244
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 0000 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF OF PF CF
DX 0000 SP FFFE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 1 0 1 0

CMD >
012D FEC9 DEC CL
012F 75E2 JNZ 0113
0131 B80000 MOV AX,0000
0134 CD21 INT 21
0136 7504 JNZ 013C
0138 85C0 TEST AX,AX
013A 741C JZ 0158
013C C746DC0000 MOV [BP-241,0000]
0141 8E5EFC MOV DS,[BP-04]

DS:0100 E9 00 00 00 00 00 00 F4
DS:0108 01 10 EB 19 EB B1 10 8B
DS:0110 16 07 01 D1 EA 73 0E A1
DS:0118 03 01 01 06 09 01 A1 05
DS:0120 01 11 06 08 01 D1 26 03
DS:0128 01 D1 16 05 01 FE C9 75
DS:0130 E2 B8 00 4C CD 21 75 04
DS:0138 85 C0 74 1C C7 46 DC 00
DS:0140 00 8E 5E FC 83 7D 0E 00
DS:0148 74 09 8B 46 F2 4B 3B 46

DS:0000 CD 20 FF 9F 00 EA FF FF AD DE 1B 05 C5 06 00 00 = f.a .+.
DS:0010 1B 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF .....f. ....
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF FF EB 19 E6 11 .....p.
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 6.....J. ....
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

1 Step 2ProcStep 3Retrieve 4help ON 5BRK Mem 6 7 up 8 dn 9 le 10 ri
```

Cx becomes 0 which raises ZF resulting the Program to end.